

INDEX

SNO	CONTENT	PAGE NO	SIGN
1.	IDENTIFICATION OF MISSING VALUES AND DETECTION OF OUTLIERS		
2.	CREATION OF SUMMARY TABLE AND VISUALIZATION OF DATA DISTRIBUTION		
3.	GENERATION OF PLOTS AND APPLICATION OF SCALING		
4.	CREATION OF A SCATTERPLOT AND INTERPRET THE RELATIONSHIP		
5.	CREATION OF A THREE-VARIABLE CONTINGENCY TABLE		
6.	TIME SERIES DATA ANALYSIS FOR CLEAN MISSING OR INCONSISTENT TIMESTAMPS		
7.	VISUALIZATION OF DATASET USING MULTIPLE SUBPLOT.		
8.	GENERATION OF CORRELATION HEATMAP AND A MAP-BASED PLOT U.		
9.	VISUALIZATION OF WORD FREQUENCIES FROM A SMALL TEXT DATASET.		
10.	CREATION OF CHART USING INTERACTIVE VISUALIZATION LIBRARIES.		

Ex. No: 1

Date:

Identification of missing values and detection of outliers

Aim:

To identify and handle missing values in a dataset to ensure data quality and to detect, analyze outliers that may affect the accuracy of data analysis or modeling.

Procedure:

- Step 1: Load the dataset into a suitable tool (Excel, Python, R, etc.)
- Step 2: Identify Missing Values
- Step 3: Visualize missing values
- Step 4: Detect Outliers using IQR
- Step 5: Boxplots to visualize outliers

Program:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("DataSet.csv")
print("Original Dataset:\n", df)

print("\nMissing Values Count:\n", df.isnull().sum())

sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title("Missing Values Heatmap")
plt.show()

def detect_outliers_iqr(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return column[(column < lower_bound) | (column > upper_bound)]

print("\nOutliers Detected:")
for col in df.select_dtypes(include=np.number).columns:
    outliers = detect_outliers_iqr(df[col].dropna())
    print(f"{col} outliers:\n{outliers}\n")

for col in df.select_dtypes(include=np.number).columns:
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()
```

OutPut:

Original Dataset:

	Age	Salary	Experience
0	25.0	50000.0	1.0
1	27.0	54000.0	3.0
2	29.0	NaN	4.0
3	NaN	58000.0	2.0
4	22.0	60000.0	1.0
5	130.0	650000.0	30.0
6	28.0	62000.0	5.0
7	32.0	59000.0	4.0
8	21.0	61000.0	3.0
9	NaN	57000.0	2.0
10	120.0	63000.0	NaN

Missing Values Count:

Age 2
Salary 1
Experience 1
dtype: int64

Outliers Detected:

Age outliers:

5 130.0
10 120.0

Name: Age, dtype: float64

Salary outliers:

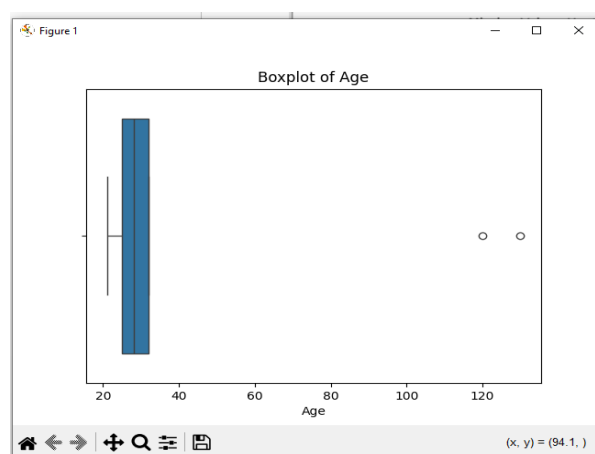
0 50000.0
5 650000.0

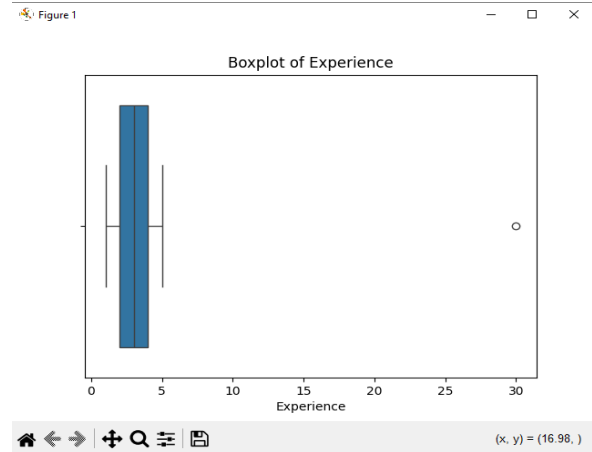
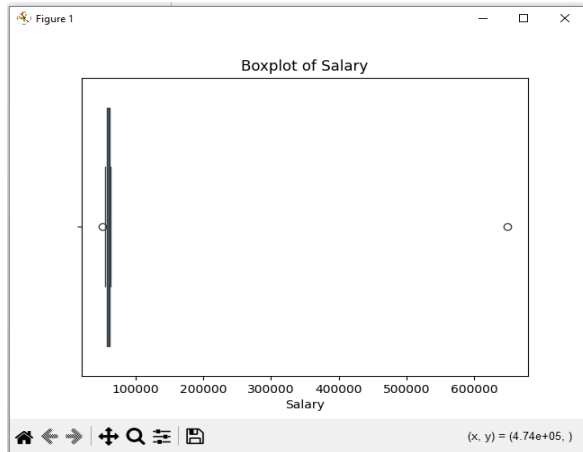
Name: Salary, dtype: float64

Experience outliers:

5 30.0

Name: Experience, dtype: float64





Result:

Successfully identified and handled missing values in a dataset to ensure data quality and detected, analyzed outliers that may affected the accuracy of data analysis or modeling.

Ex: 2 Creation of summary table and visualization of data distribution

Date:

Aim:

To create a summary table displaying descriptive statistics of a dataset and visualize the distribution of data through histograms, boxplots, and pairplots, facilitating exploratory data analysis (EDA).

Procedure:

Step1: Loads a dataset from CSV or uses Seaborn's sample dataset if file_path is none.

Step2: Display the first few rows and information about the dataset to understand its structure.

Step3: Use Pandas' describe() method to generate a summary table of descriptive statistics for all columns (numerical and categorical).

Step4: Print or return the summary table for review.

Step5: Extract columns with numerical data types for visualization purposes.

Step6: Plot histograms for each numeric column to view the frequency distribution, generate boxplots for numeric columns to detect outliers and understand spread and use Seaborn's pairplot to visualize relationships between numeric variables if the number of numeric columns is reasonable

Step7: Show the generated plots for visual analysis.

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

def load_dataset(file_path=None):
    if file_path:
        df = pd.read_csv(DataSet.csv)
    else:
        df = sns.load_dataset('tips') # Load built-in dataset
    return df

def create_summary_table(df):
    print("\nSummary Table:")
    summary = df.describe(include='all').T
    print(summary)
    return summary

def visualize_data_distribution(df):
    numeric_cols = df.select_dtypes(include='number').columns
    df[numeric_cols].hist(bins=15, figsize=(15, 6), layout=(2, -1))
    plt.suptitle("Histograms of Numeric Columns")
    plt.tight_layout()
    plt.show()
```

```

plt.figure(figsize=(15, 6))
df[numeric_cols].plot(kind='box', subplots=True, layout=(1, len(numeric_cols)), figsize=(15, 5))
plt.suptitle("Boxplots of Numeric Columns")
plt.tight_layout()
plt.show()
if len(numeric_cols) <= 5:
    sns.pairplot(df[numeric_cols])
    plt.suptitle("Pairplot of Numeric Columns", y=1.02)
    plt.show()

def main():
    df = load_dataset()
    print("Data Preview:\n", df.head())
    summary = create_summary_table(df)
    visualize_data_distribution(df)
if __name__ == "__main__":
    main()

```

Output:

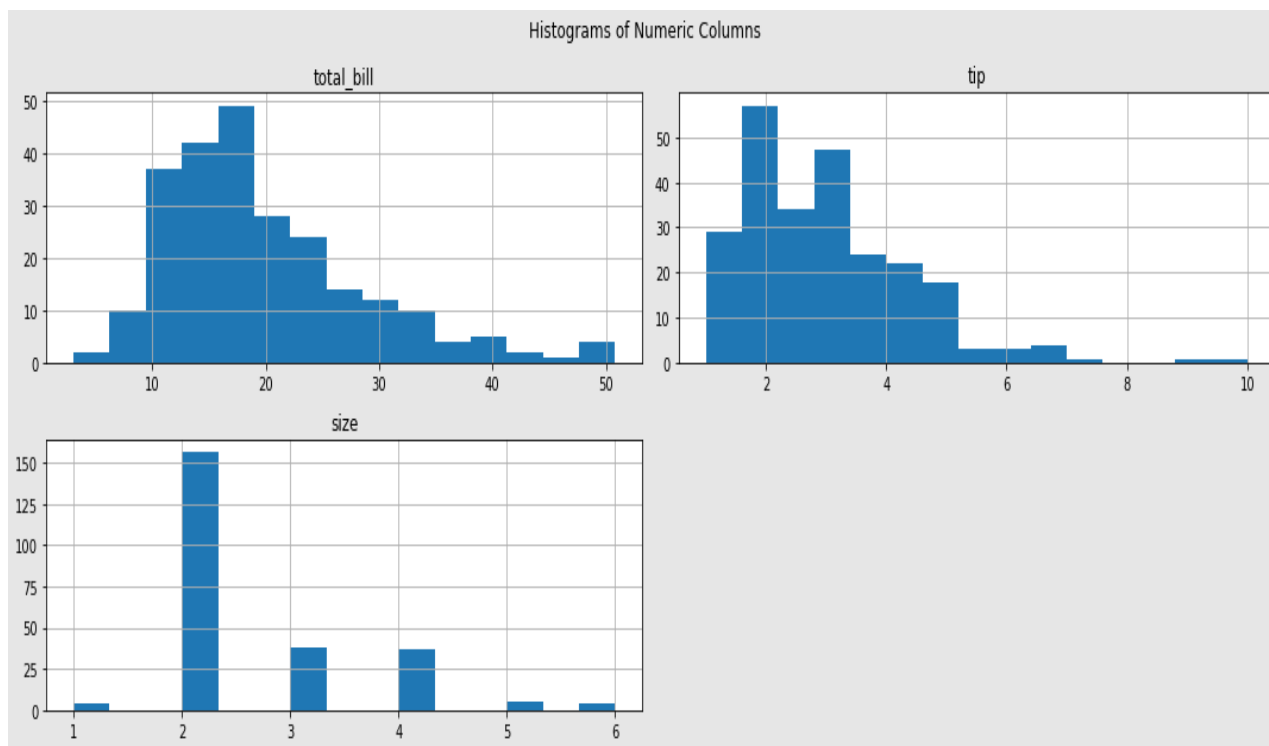
Data Preview:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

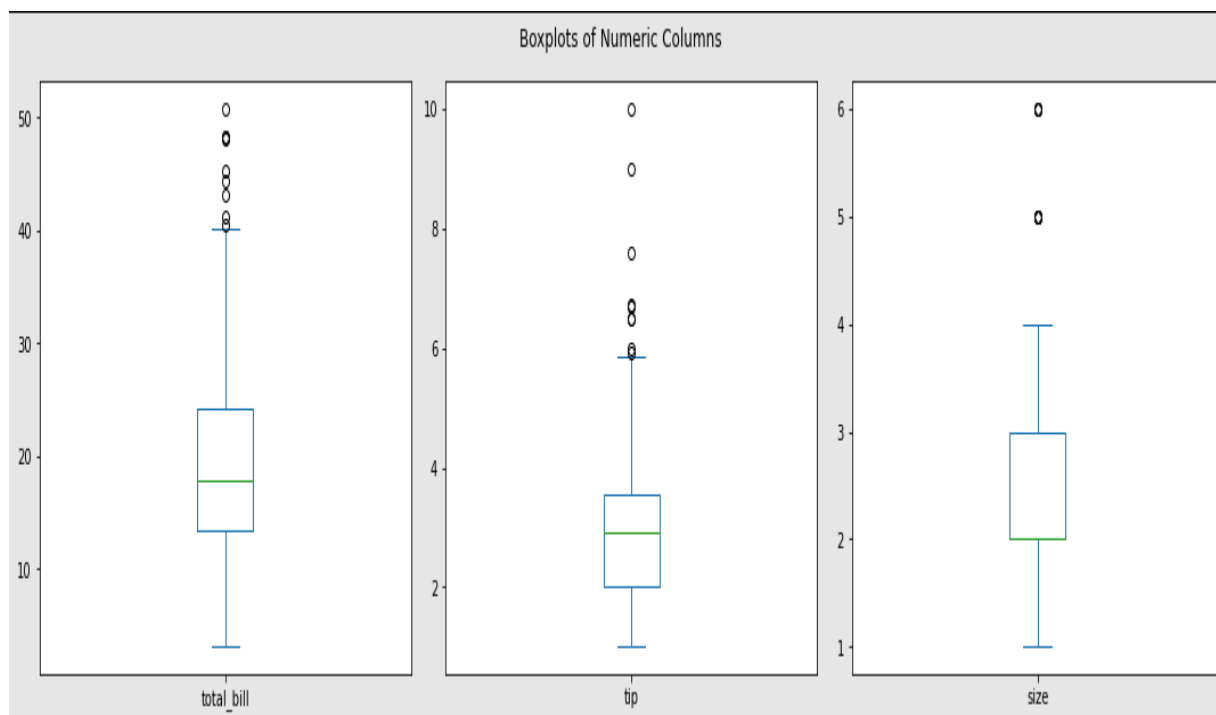
Summary Table:

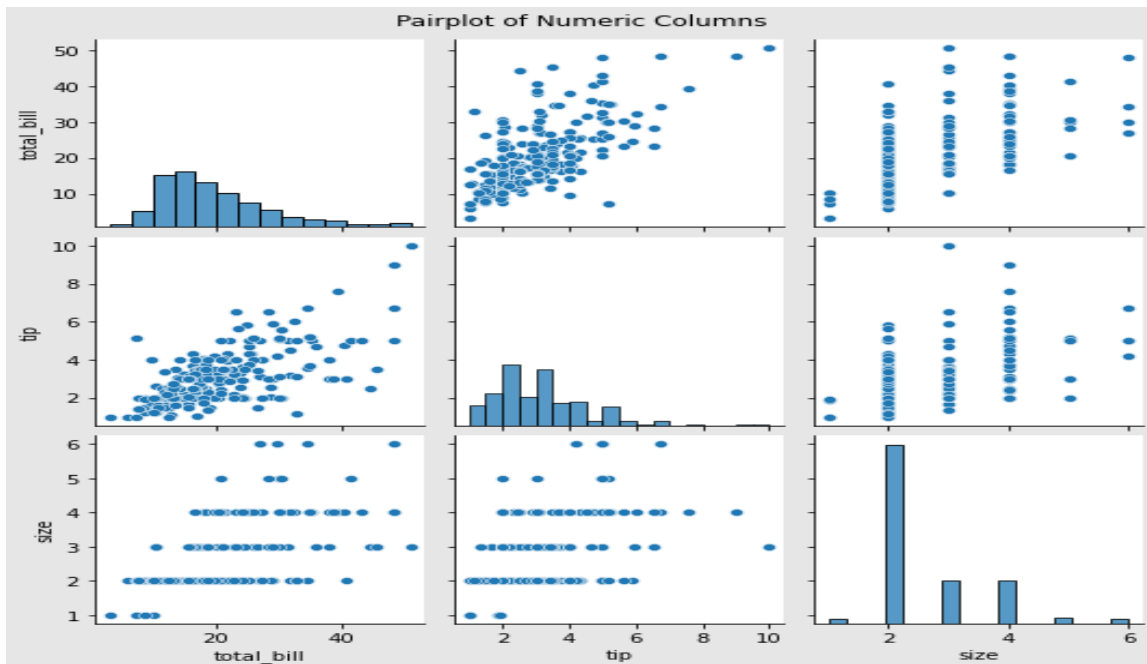
	count	unique	top	freq	mean	std	min	25%
\								
total_bill	244.0	NaN	NaN	NaN	19.785943	8.902412	3.07	13.3475
tip	244.0	NaN	NaN	NaN	2.998279	1.383638	1.0	2.0
sex	244	2	Male	157	NaN	NaN	NaN	NaN
smoker	244	2	No	151	NaN	NaN	NaN	NaN
day	244	4	Sat	87	NaN	NaN	NaN	NaN
time	244	2	Dinner	176	NaN	NaN	NaN	NaN
size	244.0	NaN	NaN	NaN	2.569672	0.9511	1.0	2.0

	50%	75%	max
total_bill	17.795	24.1275	50.81
tip	2.9	3.5625	10.0
sex	NaN	NaN	NaN
smoker	NaN	NaN	NaN
day	NaN	NaN	NaN
time	NaN	NaN	NaN
size	2.0	3.0	6.0



<Figure size 1080x432 with 0 Axes>





Result:

We have successfully created a summary table displaying descriptive statistics of a dataset and visualize the distribution of data through histograms, boxplots, and pairplots, facilitating exploratory data analysis (EDA).

Ex: 3

Generation of Plots and application of scaling

Date:

Aim:

To generate basic 2D plots using matplotlib and apply various scaling techniques such as linear scale and logarithmic scale on the axes.

Procedure:

Step1: Import the required libraries (matplotlib.pyplot and numpy).

Step2: Generate sample data for plotting (e.g., linear and exponential).

Step3: Create subplots to:

- Plot data using a linear scale on both axes.
- Apply logarithmic scaling on the x-axis.
- Apply logarithmic scaling on the y-axis.
- Apply logarithmic scaling on both axes.

Step4: Display all plots using matplotlib.

Program:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(1, 100, 500) # Avoid 0 to use log scale
y_linear = x
y_exp = np.exp(x / 20)
```

```
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
```

```
axs[0, 0].plot(x, y_linear, label="y = x")
axs[0, 0].set_title("Linear Scale (X and Y)")
axs[0, 0].set_xlabel("X")
axs[0, 0].set_ylabel("Y")
axs[0, 0].legend()
axs[0, 0].grid(True)
```

```
axs[0, 1].plot(x, y_exp, label="y = exp(x/20)")
axs[0, 1].set_xscale('log')
axs[0, 1].set_title("Logarithmic X-axis")
axs[0, 1].set_xlabel("X (log scale)")
axs[0, 1].set_ylabel("Y")
axs[0, 1].legend()
axs[0, 1].grid(True)
```

```
axs[1, 0].plot(x, y_exp, label="y = exp(x/20)")
axs[1, 0].set_yscale('log')
axs[1, 0].set_title("Logarithmic Y-axis")
axs[1, 0].set_xlabel("X")
```

```

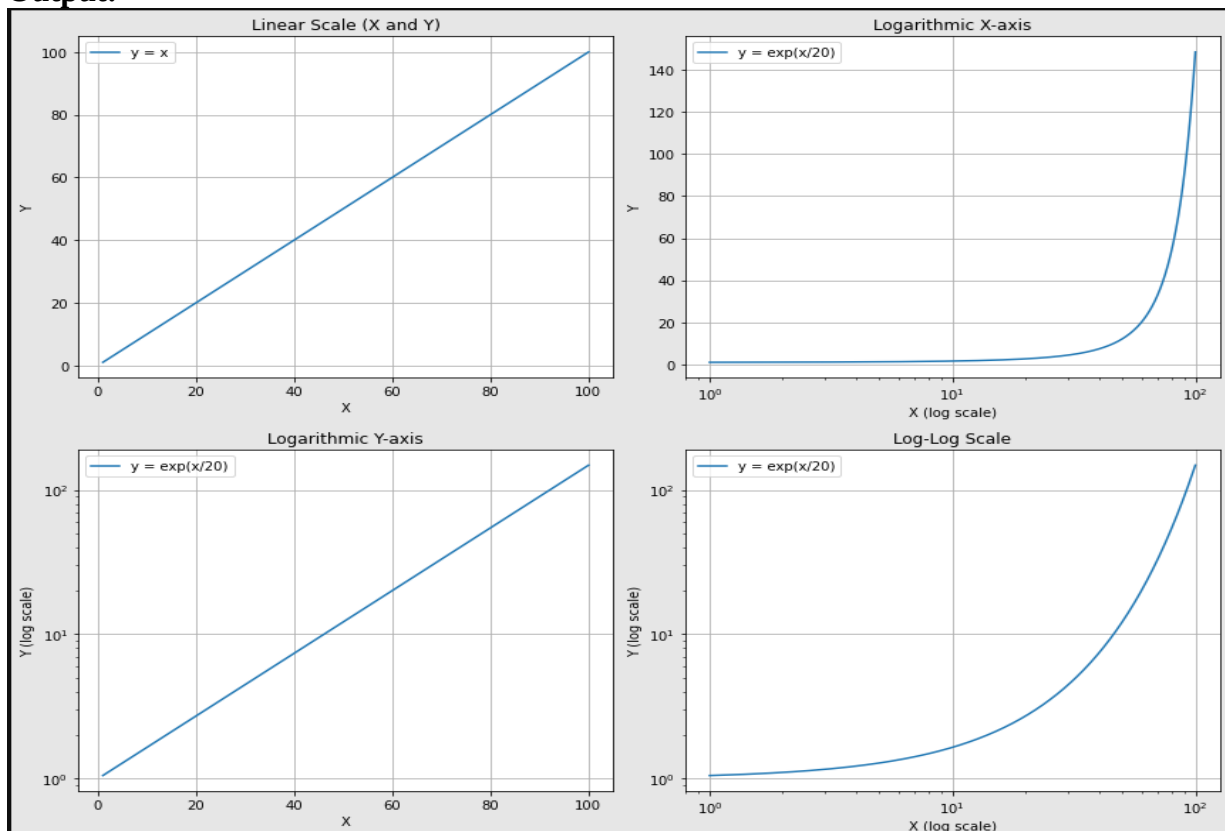
axs[1, 0].set_ylabel("Y (log scale)")
axs[1, 0].legend()
axs[1, 0].grid(True)

axs[1, 1].plot(x, y_exp, label="y = exp(x/20)")
axs[1, 1].set_xscale('log')
axs[1, 1].set_yscale('log')
axs[1, 1].set_title("Log-Log Scale")
axs[1, 1].set_xlabel("X (log scale)")
axs[1, 1].set_ylabel("Y (log scale)")
axs[1, 1].legend()
axs[1, 1].grid(True)

plt.tight_layout()
plt.show()

```

Output:



Result:

We have successfully generated basic 2D plots using matplotlib and applied various scaling techniques such as linear scale and logarithmic scale on the axes.

Ex: 4

Creation of a scatterplot and interpret the relationship

Date:

Aim:

To create a scatter plot using matplotlib and interpret the type of relationship (positive, negative, or no correlation) between two variables.

Procedure:

Step1: Import required libraries: matplotlib.pyplot and numpy.

Step2: Generate or input two sets of numerical data (e.g., height vs weight, study time vs marks).

Step3: Create a scatter plot using plt.scatter().

Step4: Label the axes and title the graph for clarity.

Step5: Interpret the type of relationship based on the pattern of points.

- The scatter plot shows that as study hours increase, exam scores tend to increase, indicating a positive linear correlation.
- The points are somewhat dispersed due to noise, but the overall upward trend is clear.
- This implies that students who study more generally achieve higher scores.

Program:

```
import matplotlib.pyplot as plt
import numpy as np

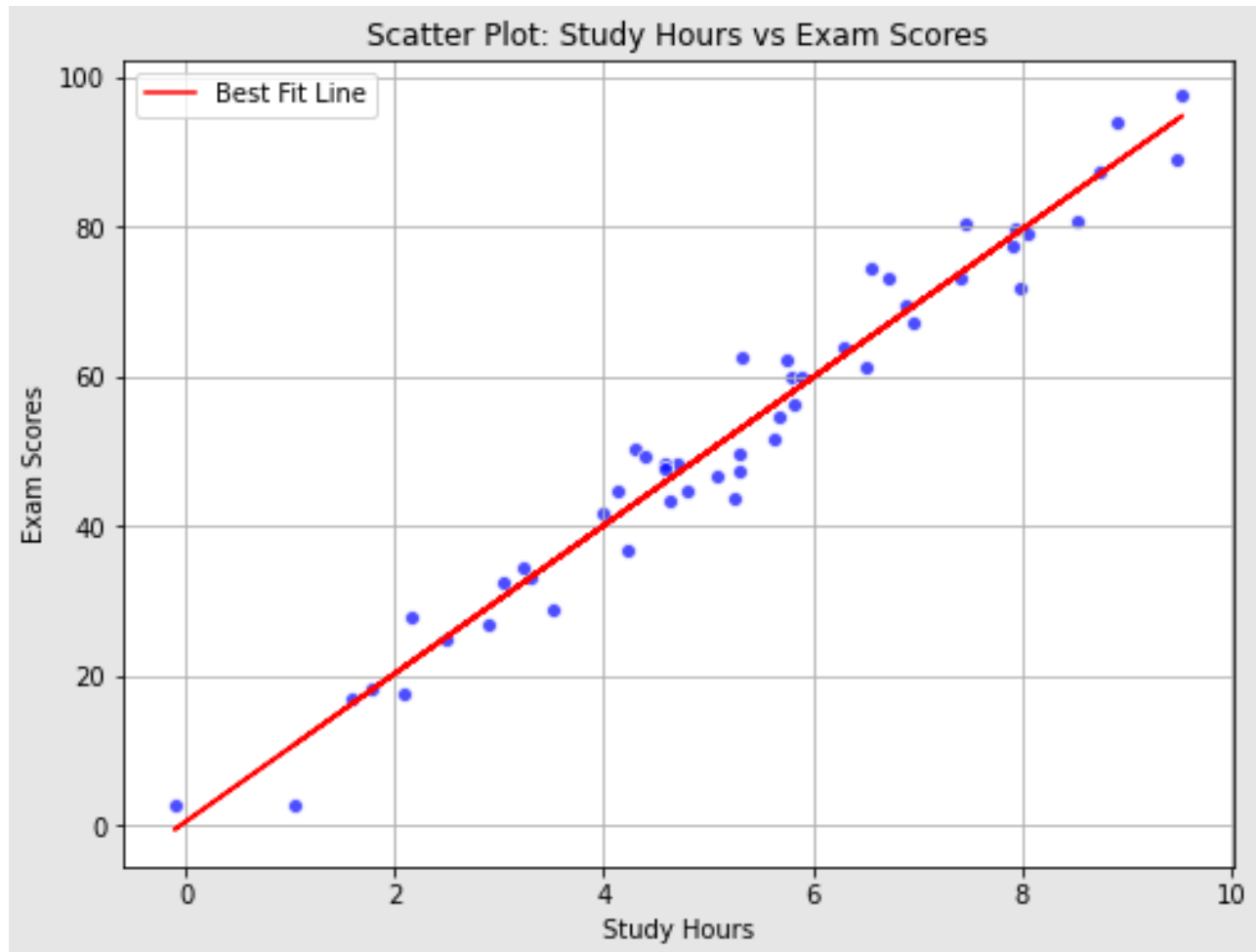
# For example: Study Hours vs Exam Scores
np.random.seed(0)
study_hours = np.random.normal(5, 2, 50) # Mean 5 hours, std dev 2
exam_scores = 10 * study_hours + np.random.normal(0, 5, 50) # Linear relation with noise

plt.figure(figsize=(8, 6))
plt.scatter(study_hours, exam_scores, color='blue', alpha=0.7, edgecolors='w')
plt.title("Scatter Plot: Study Hours vs Exam Scores")
plt.xlabel("Study Hours")
plt.ylabel("Exam Scores")
plt.grid(True)

m, b = np.polyfit(study_hours, exam_scores, 1)
plt.plot(study_hours, m*study_hours + b, color='red', label='Best Fit Line')
plt.legend()

plt.show()
```

Output:



Result:

We have successfully created a scatter plot using matplotlib and interpret the type of relationship (positive, negative, or no correlation) between two variables.

Ex: 5

Creation of a three-variable contingency table

Date:

Aim:

To create a three-variable contingency table (also known as a 3D cross-tabulation) using Python's pandas library, and to analyze the frequency distribution across three categorical variables.

Procedure:

Step1: Import the required library: pandas.

Step2: Create or load a dataset containing at least three categorical variables.

Step3: Use the `pd.crosstab()` function to compute a multi-indexed frequency table.

Step4: Interpret the result to understand the joint distribution across the three variables.

Step5: Print or return the summary table for review.

Program:

```
import pandas as pd
import numpy as np

np.random.seed(42)

data = {
    'Gender': np.random.choice(['Male', 'Female'], 100),
    'Program': np.random.choice(['Science', 'Arts', 'Commerce'], 100),
    'Grade': np.random.choice(['Pass', 'Fail'], 100)
}

df = pd.DataFrame(data)

print("Sample Data:")
print(df.head())

contingency_table = pd.crosstab(
    [df['Gender'], df['Program']], # Rows: Gender and Program
    df['Grade']                  # Columns: Grade
)

print("\nThree-Variable Contingency Table:")
print(contingency_table)
```

Output:

Sample Data:

	Gender	Program	Grade
0	Male	Commerce	Fail
1	Female	Arts	Fail
2	Male	Arts	Fail
3	Male	Arts	Fail
4	Male	Arts	Fail

Three-Variable Contingency Table:

Grade		Fail	Pass
Gender	Program		
Female	Arts	8	4
	Commerce	12	10
	Science	11	11
Male	Arts	8	6
	Commerce	7	10
	Science	7	6

Result:

We have successfully created a three-variable contingency table (also known as a 3D cross-tabulation) using Python's pandas library, and analyzed the frequency distribution across three categorical variables.

Ex: 6 Time series data analysis for clean missing or inconsistent timestamps

Date:

Aim:

To perform cleaning and preprocessing of time series data by detecting and handling missing or inconsistent timestamps, using Python's pandas library.

Procedure:

Step1: Import required libraries (pandas, numpy).

Step2: Create or load a time series dataset with intentional gaps or inconsistencies in the timestamps.

Step3: Convert the timestamp column to a proper date time format.

Step4: Set the timestamp column as the index for time series operations.

Step5: Reindex the data to a regular time frequency (e.g., hourly or daily).

Step6: Identify and fill or interpolate missing timestamps and associated data.

Step7: Print or return the summary table for review.

Program:

```
import pandas as pd
import numpy as np

rng = pd.date_range('2023-01-01', periods=10, freq='D')
rng = rng.delete([3, 6]) # Simulate missing timestamps

data = pd.DataFrame({
    'Timestamp': rng,
    'Value': np.random.randint(10, 100, size=len(rng))
})

print("Original Data (with missing timestamps):")
print(data)

data['Timestamp'] = pd.to_datetime(data['Timestamp'])
data.set_index('Timestamp', inplace=True)

full_range = pd.date_range(start=data.index.min(), end=data.index.max(), freq='D')
data_reindexed = data.reindex(full_range)

data_cleaned = data_reindexed.interpolate(method='linear') # You can also use
.fillna(method='ffill')

data_cleaned.index.name = 'Timestamp'

print("\nCleaned Data (missing timestamps filled and values interpolated):")
print(data_cleaned)
```

Output:

Original Data (with missing timestamps):

	Timestamp	Value
0	2023-01-01	98
1	2023-01-02	80
2	2023-01-03	36
3	2023-01-05	65
4	2023-01-06	96
5	2023-01-08	18
6	2023-01-09	74
7	2023-01-10	75

Cleaned Data (missing timestamps filled and values interpolated):

	Timestamp	Value
	2023-01-01	98.0
	2023-01-02	80.0
	2023-01-03	36.0
	2023-01-04	50.5
	2023-01-05	65.0
	2023-01-06	96.0
	2023-01-07	57.0
	2023-01-08	18.0
	2023-01-09	74.0
	2023-01-10	75.0

Result:

We have successfully performed cleaning and preprocessing of time series data by detecting and handling missing or inconsistent timestamps, using Python's pandas library.

Ex: 7

Visualization of dataset using multiple subplot

Date:

Aim:

To visualize different aspects of a dataset using multiple subplots in a single figure, allowing easy comparison and analysis of multiple variables simultaneously.

Procedure:

Step1: Import the required libraries (pandas, matplotlib.pyplot, seaborn, and optionally numpy).

Step2: Load or create a sample dataset with multiple variables.

Step3: Create a figure with subplots using plt.subplots().

Step4: Plot different types of charts (line plot, bar chart, histogram, scatter plot) in each subplot.

Step5: Format titles, axes, and layout for clarity.

Step6: Display the combined visualization.

Step7: Show the generated plots for visual analysis.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(1)
n = 100
df = pd.DataFrame({
    'Age': np.random.randint(22, 60, size=n),
    'Salary': np.random.randint(30000, 100000, size=n),
    'Experience': np.random.randint(1, 35, size=n),
    'Department': np.random.choice(['HR', 'IT', 'Finance', 'Sales'], size=n)
})

fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Multiple Subplot Visualization of Dataset', fontsize=16)

axs[0, 0].hist(df['Age'], bins=10, color='skyblue', edgecolor='black')
axs[0, 0].set_title('Distribution of Age')
axs[0, 0].set_xlabel('Age')
axs[0, 0].set_ylabel('Frequency')

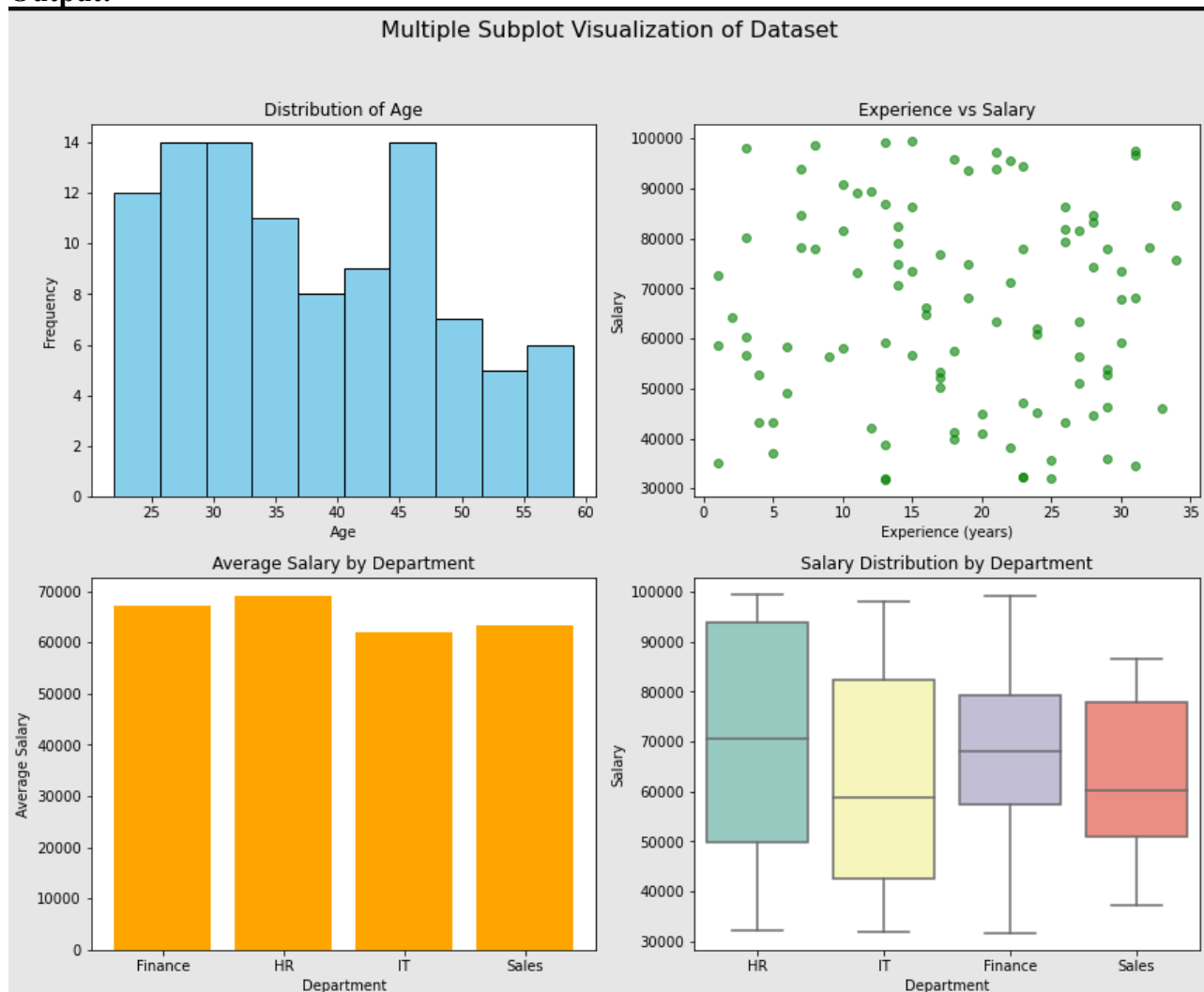
axs[0, 1].scatter(df['Experience'], df['Salary'], color='green', alpha=0.6)
axs[0, 1].set_title('Experience vs Salary')
axs[0, 1].set_xlabel('Experience (years)')
axs[0, 1].set_ylabel('Salary')

avg_salary = df.groupby('Department')['Salary'].mean()
axs[1, 0].bar(avg_salary.index, avg_salary.values, color='orange')
axs[1, 0].set_title('Average Salary by Department')
axs[1, 0].set_xlabel('Department')
axs[1, 0].set_ylabel('Average Salary')

sns.boxplot(x='Department', y='Salary', data=df, ax=axs[1, 1], palette='Set3')
axs[1, 1].set_title('Salary Distribution by Department')

plt.tight_layout(rect=[0, 0, 1, 0.95]) # Leave space for the main title
plt.show()
```

Output:



Result:

We have successfully visualized different aspects of a dataset using multiple subplots in a single figure, allowing easy comparison and analysis of multiple variables simultaneously.

Ex: 8 **Generation of correlation heatmap and a map-based plot u**
Date:

Aim:

To generate a correlation heatmap for understanding relationships between numerical variables, and to create a map-based plot to visualize spatial data using Python libraries like seaborn, pandas, and plotly.

Procedure:

Step1: Import necessary libraries: pandas, seaborn, matplotlib.

Step2: Create or load a dataset with multiple numerical columns.

Step3: Calculate the correlation matrix using `DataFrame.corr()`.

Step4: Use `seaborn.heatmap()` to visualize the correlation matrix.

Step5: Map-Based Plot: Use a dataset with geospatial information (e.g., latitude and longitude).

Step6: Use `plotly.express.scatter_geo()` or `folium.Map()` to create a map.

Step7: Customize markers and layout.

Program:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px

np.random.seed(0)
df = pd.DataFrame({
    'Age': np.random.randint(18, 60, 100),
    'Salary': np.random.randint(30000, 100000, 100),
    'Experience': np.random.randint(1, 40, 100),
    'Score': np.random.rand(100) * 100
})

corr_matrix = df.corr()

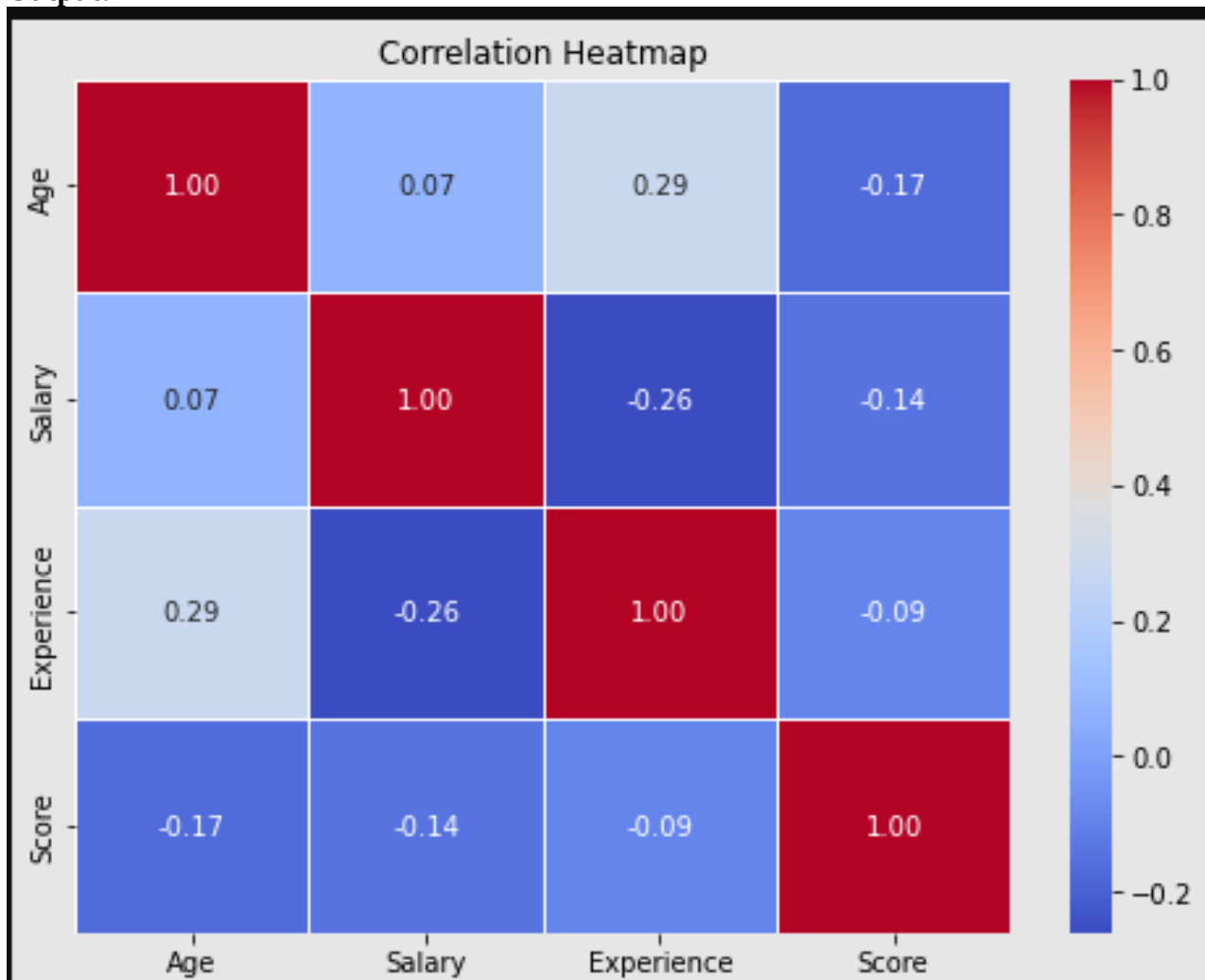
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()

map_data = pd.DataFrame({
    'City': ['Delhi', 'Mumbai', 'Bangalore', 'Chennai', 'Kolkata'],
    'Latitude': [28.61, 19.07, 12.97, 13.08, 22.57],
    'Longitude': [77.20, 72.87, 77.59, 80.27, 88.36],
    'Population': [19000000, 20000000, 12000000, 10000000, 15000000]
})

fig = px.scatter_geo(map_data,
                    lat='Latitude',
                    lon='Longitude',
                    text='City',
                    size='Population',
                    projection='natural earth',
                    title='Map-Based Plot of Indian Cities by Population')

fig.show()
```

Output:



Result:

We have successfully generated a correlation heatmap for understanding relationships between numerical variables, and to created a map-based plot to visualize spatial data using Python libraries like seaborn, pandas, and plotly.

Ex: 9 **Visualization of word frequencies from a small text dataset.**

Date:

Aim:

To extract and visualize the frequency of words from a small text dataset using Python libraries like collections, matplotlib, and wordcloud.

Procedure:

Step1: Import required libraries: collections, matplotlib, and wordcloud.

Step2: Define or load a small sample text dataset.

Step3: Preprocess the text:

Step4: Convert to lowercase.

Step5: Remove punctuation and stopwords.

Step6: Tokenize the text and count word frequencies using collections.Counter.

Step7: Visualize the word frequencies using:

Step8: A bar plot for top N words.

Step9: A word cloud for an intuitive visual.

Program:

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter
import string

text = """
Data science is an interdisciplinary field that uses scientific methods, processes, algorithms
and systems to extract knowledge and insights from structured and unstructured data.
Data science is related to data mining, machine learning and big data.
"""

text = text.lower().translate(str.maketrans("", "", string.punctuation))

words = text.split()

stopwords = set([
    'is', 'an', 'that', 'and', 'to', 'from', 'the', 'a', 'of', 'in', 'on', 'for', 'with', 'uses'
])
filtered_words = [word for word in words if word not in stopwords]

word_freq = Counter(filtered_words)

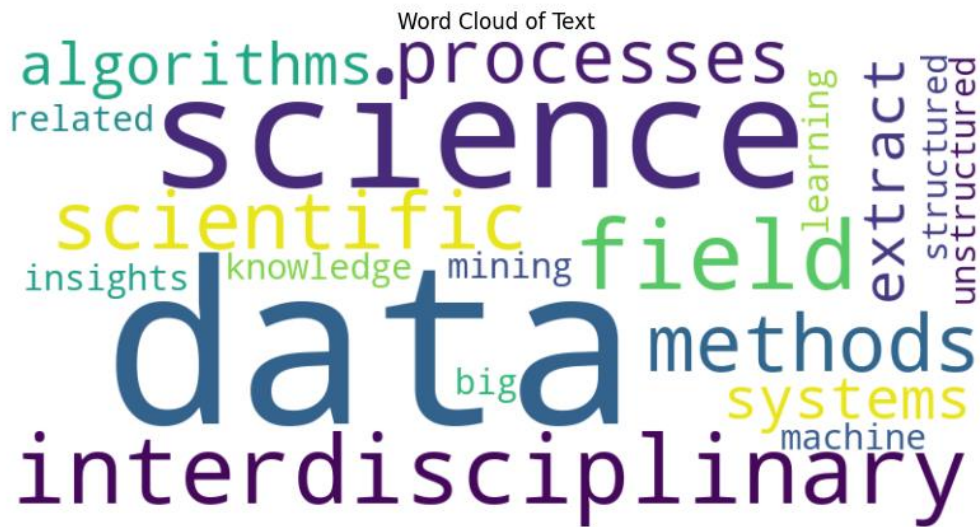
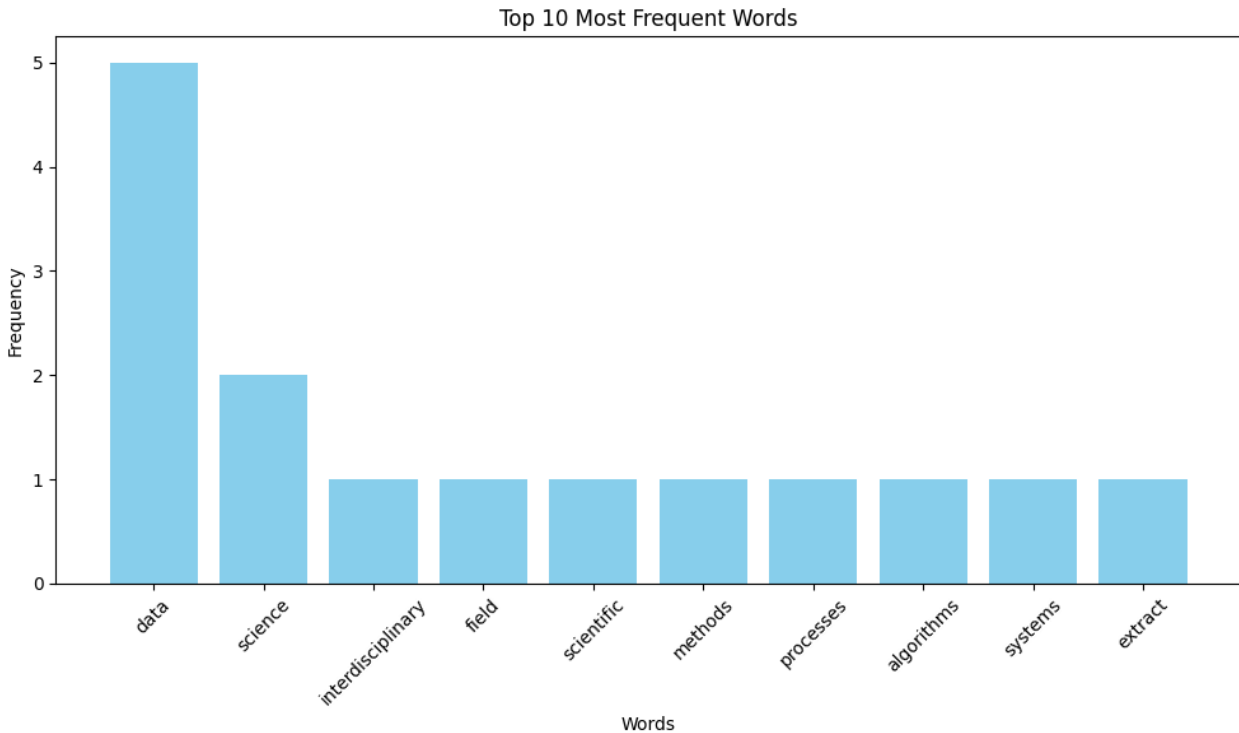
most_common_words = word_freq.most_common(10)
words_list, freq_list = zip(*most_common_words)

plt.figure(figsize=(10, 6))
plt.bar(words_list, freq_list, color='skyblue')
plt.title("Top 10 Most Frequent Words")
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

wordcloud = WordCloud(width=800, height=400, background_color='white').generate(
    '.join(filtered_words))

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Word Cloud of Text")
plt.show()
```

Output:



Result:

We have successfully extracted and visualized the frequency of words from a small text dataset using Python libraries like collections, matplotlib, and wordcloud.

Ex: 10 **Creation of chart using interactive visualization libraries.**

Date:

Aim:

To create interactive charts using Python's visualization libraries, specifically Plotly, in order to enhance data exploration, interactivity, and user engagement compared to static plots.

Procedure:

Step1: Import the required library: `plotly.express`.

Step2: Load or create a sample dataset with numeric and categorical variables.

Step3: Create various interactive charts such as: 1. Line Chart, 2. Bar Chart, 3. Scatter Plot, 4. Pie Chart

Step4: Customize interactivity features: hover tooltips, zoom, legends, etc.

Step5: Display the charts in a browser or notebook environment.

Program:

```
import plotly.express as px
import pandas as pd
import numpy as np

np.random.seed(42)
df = pd.DataFrame({
    'Year': np.arange(2015, 2025),
    'Sales': np.random.randint(200, 1000, 10),
    'Profit': np.random.randint(50, 500, 10),
    'Region': np.random.choice(['North', 'South', 'East', 'West'], size=10)
})

fig_line = px.line(df, x='Year', y='Sales', title='Interactive Line Chart - Sales Over Years',
markers=True)
fig_line.show()

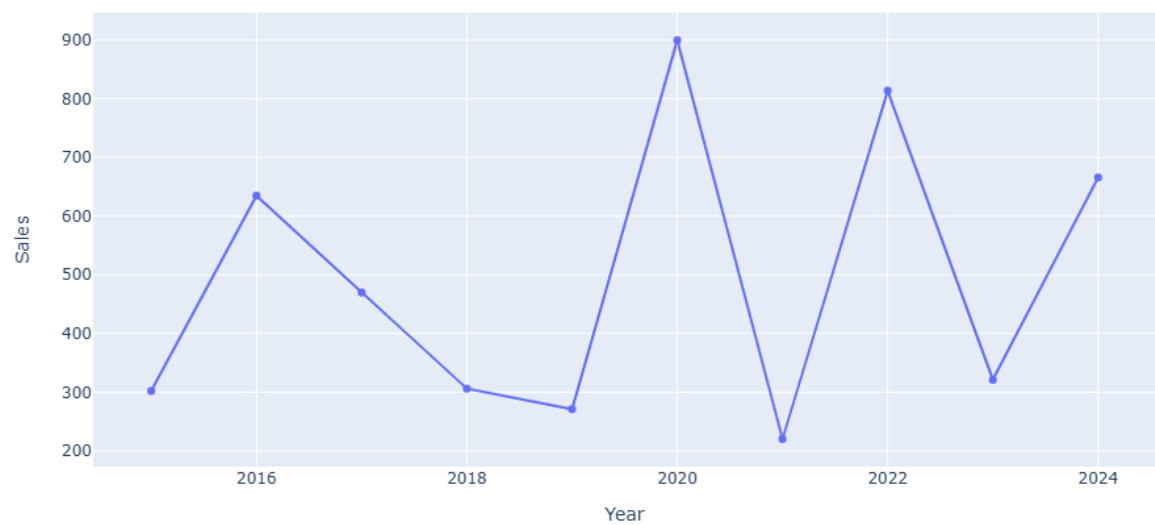
fig_bar = px.bar(df, x='Year', y='Profit', color='Region', title='Interactive Bar Chart - Profit by
Region')
fig_bar.show()

fig_scatter = px.scatter(df, x='Sales', y='Profit', size='Profit', color='Region',
title='Interactive Scatter Plot - Sales vs Profit')
fig_scatter.show()

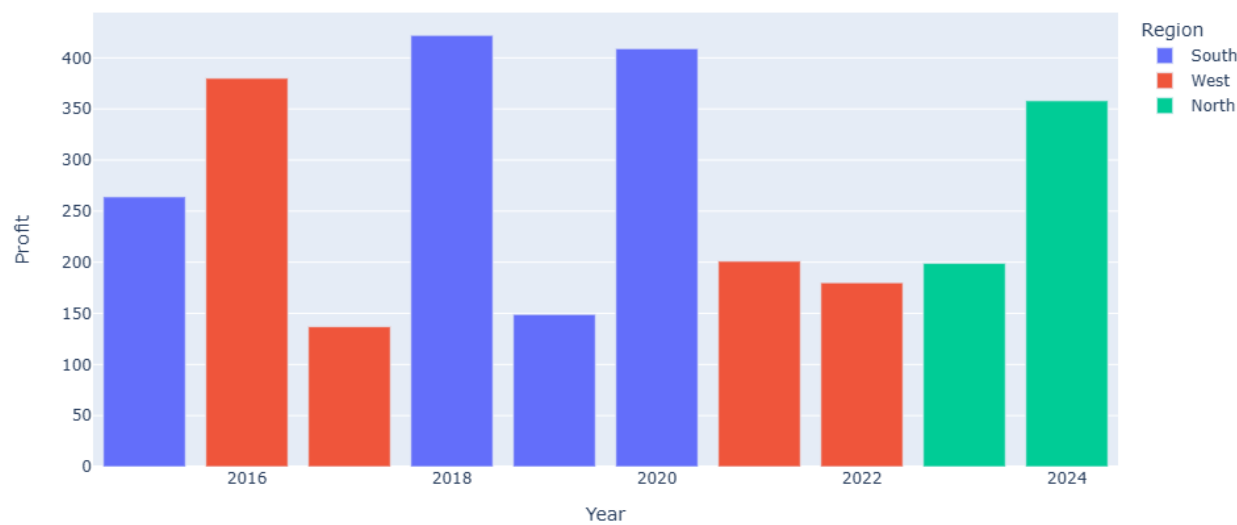
region_summary = df.groupby('Region').sum(numeric_only=True).reset_index()
fig_pie = px.pie(region_summary, names='Region', values='Sales', title='Sales Distribution by
Region')
fig_pie.show()
```

Output:

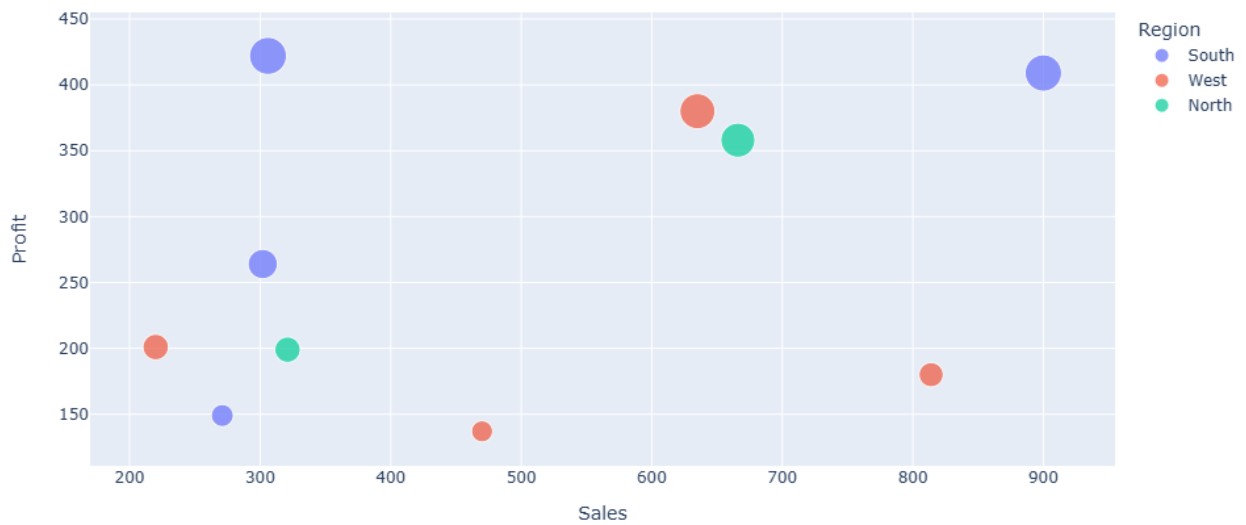
Interactive Line Chart - Sales Over Years



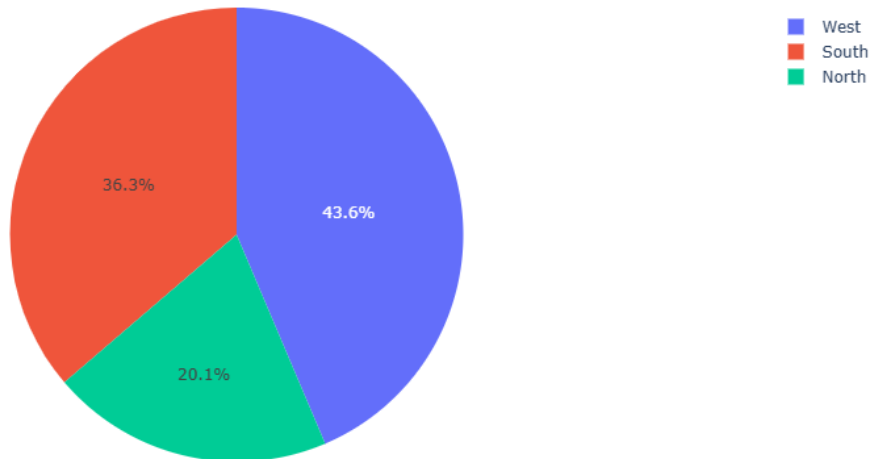
Interactive Bar Chart - Profit by Region



Interactive Scatter Plot - Sales vs Profit



Sales Distribution by Region



Result:

We have successfully created interactive charts using Python's visualization libraries, specifically Plotly, in order to enhance data exploration, interactivity, and user engagement compared to static plots.